Architecture and Methodology

Simplified Lesk Word Sense Disambiguation for Closed-Caption Disambiguation

MT-AIML Mini Project | Theoretical concepts, mathematical formulation and system design | October 2026

Contents

1. Introduction and Scope 2. Problem Formulation 3. Theoretical Background 4. System Architecture 5. Text Preprocessing Design 6. Sense Inventory and Signature Construction 7. Scoring Model and Decision Rule 8. Baseline Model 9. Evaluation Methodology 10. Complexity Analysis 11. Design Rationale, Limitations and Extensions Appendix A: Notation Appendix B: Configuration Summary

1. Introduction and Scope

Closed-captioning systems convert speech into text that is displayed, and often translated, in real time. A persistent difficulty is **lexical ambiguity**: the same surface word can carry several unrelated meanings, and a translation system must know which one the speaker intended. The word *bank* in “He deposited money in the bank” and in “The fisherman sat on the bank of the river” is written identically but translates differently in almost every target language.

This document describes the architecture and methodology of a lightweight, interpretable **Word Sense Disambiguation (WSD)** system based on the **Simplified Lesk** algorithm. The system selects, for a given sentence and target word, the most appropriate sense from the WordNet sense inventory by measuring lexical overlap between the target’s context and the textual description of each candidate sense. It is evaluated against NLTK’s built-in Lesk implementation on an annotated test set.

The document covers the theoretical concepts the system relies on, the mathematical definition of every computation, and the design of each software component (`preprocessing.py`, `models.py` and `pipeline.py`). Experimental findings and error analysis belong to the separate Evaluation Report; only the evaluation *method* is specified here.

2. Problem Formulation

Let a sentence be a sequence of tokens $X = (x_1, x_2, \ldots, x_n)$ and let w be a target word that occurs in X at position i. Let $\Sigma(w)$ denote the set of senses that the sense inventory (WordNet) lists for w. The WSD task is to find the function

$$f : (X, w) \rightarrow \hat{s} \in \Sigma(w) \tag{1}$$

such that $\hat{s}$ is the sense intended by the speaker or author. In this project the inventory is Princeton WordNet accessed through NLTK, a sense is a *synset* identified by a name of the form `lemma.pos.NN` (for example `bank.n.01`), and the supervision signal for evaluation is a single gold synset per test instance.

The method is **knowledge-based** and **unsupervised**: it needs no sense-tagged training corpus and learns no parameters. All evidence comes from the dictionary-style information stored in WordNet and from the sentence itself. The only tunable quantities are a small number of hand-set hyper-parameters (context window width and three scoring weights, Section 7).

3. Theoretical Background

3.1 Lexical ambiguity: polysemy and homonymy

A word is **polysemous** when it has several related senses (*bank* as an institution and as the building that houses it) and a **homonym** when the senses are unrelated and merely share a spelling (*bat* the animal and *bat* the sports implement). WSD systems treat both uniformly: they must choose one entry from a list of candidate senses.

3.2 WordNet as the sense inventory

WordNet is a lexical database in which words are grouped into sets of cognitive synonyms called **synsets**. Each synset represents one concept and carries the following information that this project uses:

- a **gloss** (definition), a short natural-language description of the concept;
- zero or more **example sentences** showing the concept in use;
- a set of **lemmas** (the synonyms that can express the concept), each with a corpus **frequency count** derived from sense-tagged text;
- a **part of speech**: noun (n), verb (v), adjective (a) or adverb (r).

A word form such as *bank* belongs to many synsets, ordered in WordNet roughly by decreasing frequency of use. The set Σ(w) of candidate senses is therefore the set of synsets that contain w as a lemma, optionally restricted to one part of speech.

3.3 The Lesk family of algorithms

Lesk (1986) proposed that the correct sense of an ambiguous word is the one whose dictionary definition shares the most words with the definitions of its neighbours. For two words w_1 and w_2 with candidate senses s_1 and s_2, the **original Lesk** score is

$$Overlap(s_1, s_2) = |\, gloss(s_1) \cap gloss(s_2) \,| \tag{2}$$

and the senses maximising it are chosen jointly. Because every pair of candidate senses across all words must be compared, the cost grows combinatorially with the number of ambiguous words in the window.

Simplified Lesk (Kilgarriff and Rosenzweig, 2000) removes the combinatorial step. Instead of comparing definitions with other definitions, it compares each candidate sense directly with the *words of the context*:

$$\hat{s} = \operatorname{argmax}_{s \in \Sigma(w)} |\, C \cap signature(s) \,| \tag{3}$$

Here C is the set of context words around the target and signature(s) is the bag of words describing sense s. The cost is linear in the number of candidate senses, which is what makes the method suitable for real-time captioning. This project implements a *weighted*, multi-field generalisation of Equation (3), described in Section 7.

3.4 Why overlap carries sense information

The underlying assumption is the **distributional hypothesis** as applied to dictionaries: words that co-occur with a target word in running text tend to be the words used to define or illustrate the sense in which it is used. A context containing *deposit* and *money* overlaps strongly with the gloss of the financial sense of *bank*, while a context containing *river* overlaps with the gloss of the sloping-land sense. Equation (3) is a crude but transparent estimator of the conditional preference P(s | context) based on shared vocabulary.

3.5 Reference baselines

Two reference points are useful when interpreting accuracy. A **random baseline** picks uniformly among candidates, so its expected accuracy over N instances is

$$E[Acc_{rand}] = (1 / N) \sum_{j=1..N} 1 / |\Sigma(w_j)| \quad (4)$$

A **first-sense baseline** always returns the first synset in WordNet order, which exploits the frequency ordering of senses. A knowledge-based method is only useful if it beats such trivial strategies on the target data.

4. System Architecture

The system is organised as three Python modules with a strict one-way dependency: the evaluation pipeline calls the models, and the models call the preprocessing functions. No module depends on one that sits above it. This separation makes each layer replaceable (for example, swapping the tokenizer or the scoring function) without touching the others.

Layer	Module	Responsibility
Preprocessing	`preprocessing.py`	Resource download, text cleaning, sentence segmentation, tokenization, target localisation, context-window extraction, stop-word removal, POS tagging and mapping, lemmatization, signature normalisation.
Models	`models.py`	`SimplifiedLesk` (custom weighted algorithm) and `NLTKLeskModel` (baseline wrapper); candidate retrieval, signature building, scoring, ranking, result display.
Pipeline	`pipeline.py`	Dataset loading and validation, gold-synset resolution, running both models, correctness checking, accuracy metrics, console reporting, CSV export.

4.1 Processing stages for a single instance

For one input pair (sentence, target word) the custom model executes the following ordered stages. Each stage consumes only the output of the previous one.

1. **Cleaning and tokenization.** The sentence is lower-cased, stripped of punctuation and split into word tokens.
2. **Target localisation.** The position i of the first token equal to the target word is found *before* any token is removed, so that the index is never shifted.
3. **POS detection.** The full sentence is part-of-speech tagged; the tag at position i is mapped to a WordNet category.
4. **Context extraction.** A symmetric window of half-width k is taken around i, excluding the target token itself.
5. **Context normalisation.** Stop words are removed and the remaining tokens are lemmatized using their own POS tags, yielding the context set C.
6. **Candidate retrieval.** WordNet synsets of the target are retrieved, filtered by the detected POS (with a fallback to all senses).
7. **Signature construction.** For each candidate, three token sets are built from its gloss, its examples and its lemma names.
8. **Weighted scoring.** The overlap of C with each token set is computed and combined into a single score.

9. **Ranking and decision.** Candidates are sorted by a lexicographic key and the first is returned together with its evidence (overlapping words per field).

The same instance is then passed to the NLTK baseline, and both predictions are compared with the gold synset by the pipeline (Section 9).

5. Text Preprocessing Design

Lexical overlap is only meaningful if the context and the sense signatures are expressed in the same normalised vocabulary. The preprocessing layer produces that common representation.

5.1 Cleaning and tokenization

Cleaning applies two regular-expression substitutions to the lower-cased text: every character that is not a letter, digit, whitespace or apostrophe is replaced by a space, and runs of whitespace are collapsed:

```
text = text.lower()
text = re.sub(r"[^a-z0-9\s']", " ", text)
text = re.sub(r"\s+", " ", text).strip()
```

The cleaned string is split into tokens with the NLTK word tokenizer (Punkt models). A sentence segmenter is also provided for multi-sentence caption streams, so that disambiguation can be run per sentence.

5.2 Target localisation before filtering

The token index i of the target is determined on the *unfiltered* token list by exact, case-insensitive match. Locating the target first is a deliberate design choice: removing stop words first would shift every later index and could move the window away from the intended word.

5.3 Context window

Let $t_1, \ldots, t_n$ be the tokens, i the target index and k the window half-width. The raw context is

$$C_{raw} = \{\, t_j : max(1, i - k) \le j \le min(n, i + k),\ \ j \ne i \,\} \tag{5}$$

The window is therefore symmetric, contains at most 2k tokens and is truncated at the sentence boundaries. The default half-width of the model class is k = 7; the evaluation pipeline runs with k = 5 (Appendix B). A narrow window limits noise from distant words; a wide window captures more topical evidence but also more unrelated vocabulary.

5.4 Target-word leakage protection

The target position is excluded from C_{raw} (the condition $j \ne i$ in Equation 5). If the target were retained, its own lemma would appear in every synset signature that lists it as a synonym, inflating all scores equally and obscuring the real evidence. Exclusion is positional: it removes the occurrence that was located, not every repeated occurrence of the same string in the sentence.

5.5 Stop-word removal

Tokens that belong to the NLTK English stop-word list are discarded from the context and from gloss and example text. Function words such as *the*, *of* and *to* occur in nearly every definition; counting them as matches would add the same constant to all candidates and dilute the discriminative content words.

5.6 Part-of-speech tagging and mapping

The whole cleaned sentence is tagged with the NLTK averaged-perceptron tagger, so the tagger sees the full syntactic frame of the target. The Penn Treebank tag of the target is mapped to a WordNet category by its first letter:

Treebank prefix	Meaning	WordNet tag
J (JJ, JJR, JJS)	adjective	a
V (VB, VBD, VBG, …)	verb	v
N (NN, NNS, NNP, …)	noun	n
R (RB, RBR, RBS)	adverb	r
any other tag	no category	None (no filter applied)

5.7 Lemmatization

Lemmatization maps an inflected form to its dictionary form with the WordNet lemmatizer (*flew* → *fly* when tagged as a verb; *rivers* → *river*). It lets a context word match a gloss word that differs only by inflection. Two lemmatization modes exist: context tokens are lemmatized with *their own* POS tags, while gloss and example tokens are lemmatized with the *target's* POS as the hint. Lemma names used as synonyms are normalised with the default (noun) lemmatizer.

5.8 Signature normalisation

After the steps above, every token list passes through a final filter that keeps only purely alphanumeric lower-case strings (`^[a-z0-9]+$`), removing residual fragments such as apostrophe contractions or empty strings.

6. Sense Inventory and Signature Construction

6.1 Candidate senses

Given the target w and detected POS π (which may be undefined), the candidate set is

$$\Sigma(w, \pi) = \{\, s \in \mathit{WordNet} : w \in \mathit{lemmas}(s) \wedge \mathit{pos}(s) = \pi \,\} \tag{6}$$

If π is undefined, or if filtering by π leaves no candidates (the POS tagger may disagree with WordNet's coverage), the filter is dropped and Σ(w, π) becomes the set of all senses of w. If WordNet has no entry at all, the model returns an empty prediction with score 0.

Why POS filtering matters. For a word like *watch*, noun senses (a timepiece) and verb senses (to observe) compete in an unfiltered inventory even though the sentence syntax already rules one family out. Filtering reduces |Σ|, removes impossible candidates and therefore lowers the chance of a spurious overlap selecting a sense of the wrong category.

6.2 Sense signature

The **signature** of a candidate sense is not one bag of words but a triple of separate token sets, so that evidence from different fields can be weighted differently:

$$\mathit{Sig}(s) = (\, D(s),\ \ E(s),\ \ Y(s) \,) \tag{7}$$

- **D(s)**: the normalised tokens of the synset gloss (definition).

- **E(s)**: the normalised tokens of all example sentences of the synset, concatenated. Examples show typical co-occurring vocabulary and are usually richer evidence than the short gloss.
- **Y(s)**: the normalised tokens of the lemma names of the synset (its synonyms). Underscores in multi-word lemma names are replaced by spaces before processing. A context word that is itself a synonym of the sense is direct lexical evidence for it.

Each of D, E and Y is treated as a *set*: repeated words inside one field are counted once. Synonym tokens can be switched off with the `use_synonyms` option, which makes Y(s) empty.

7. Scoring Model and Decision Rule

7.1 Field-wise overlap

Let C be the final context set from Section 5. For each candidate s the overlap with each signature field is the size of the set intersection:

$$\begin{aligned} o_D(s) &= |\, C \cap D(s) \,| \\ o_E(s) &= |\, C \cap E(s) \,| \\ o_Y(s) &= |\, C \cap Y(s) \,| \end{aligned} \tag{8}$$

7.2 Weighted Lesk score

The scores are combined linearly with non-negative weights:

$$Score(s) = \alpha \cdot o_D(s) + \beta \cdot o_E(s) + \gamma \cdot o_Y(s) \tag{9}$$

The default weights are $\alpha = 1.0$, $\beta = 2.0$ and $\gamma = 0.5$. Their ordering encodes the following reasoning:

- $\beta > \alpha$: a context word that appears in an *example sentence* has been observed in actual use of the sense, which is stronger evidence than membership in a terse definition;
- α is the neutral reference weight, equal to the classical Lesk count;
- $\gamma < \alpha$: lemma names are few and often shared by many senses of the same word, so synonym matches are useful but are given the lowest weight.

Setting $\alpha = 1$ and $\beta = \gamma = 0$ recovers a pure gloss-overlap Simplified Lesk, so the model is a strict generalisation of Equation (3).

7.3 Decision rule with deterministic tie-breaking

Several senses frequently obtain the same score, particularly when the context is short or when no overlap exists at all. To make the output deterministic and linguistically sensible, candidates are ranked by a four-component lexicographic key, compared left to right:

$$\kappa(s) = (\, Score(s),\quad o_E(s),\quad o_D(s),\quad f(s) \,) \tag{10}$$

where f(s) is the WordNet corpus frequency of the target word in sense s:

$$f(s) = \sum_{l \in lemmas(s),\, name(l) = w} count(l) \tag{11}$$

The predicted sense is the candidate with the largest key:

$$\hat{s} = \operatorname{argmax}_{s \in \Sigma(w,\, \pi)} \kappa(s) \tag{12}$$

The ranking is performed with a stable sort, so if even f(s) is tied the candidate that appears first in WordNet order (the more frequent, more general sense) wins. The rule therefore has the following behaviour:

- If evidence exists, the sense with the highest weighted overlap wins.
- If two senses have equal score, the one supported by more *example* matches wins, then the one with more *gloss* matches.
- If no sense has any overlap, all scores are 0 and the decision degenerates to a **most-frequent-sense** choice using f(s). This is the standard fallback recommended in the WSD literature when context gives no evidence.

7.4 Evidence reporting (interpretability)

For every candidate the model keeps the overlapping words of each field (O_D, O_E, O_Y), their union, the per-field partial scores and f(s). The pipeline stores the union of the winning sense's overlap words, so every prediction can be explained in words. This is the principal advantage of the method over neural WSD.

7.5 Worked example

Consider the sentence "He went to the bank to deposit his money" with target *bank*. The target is tagged as a noun, so only noun synsets are scored. After stop-word removal and lemmatization the context set (with k = 5) contains the content words *go*, *deposit* and *money*. The financial-institution synset of *bank* contains *deposit* and *money* in its gloss, giving $o_D = 2$ and, with $\alpha = 1$, Score = 2.0. Sense candidates describing sloping land share no word with the context and score 0. The financial synset is therefore ranked first on Equation (10), and the reported evidence is the overlap set {deposit, money}.

8. Baseline Model

The reference system is the Lesk implementation shipped with NLTK (`nltk.wsd.lesk`), wrapped in the class `NLTKLeskModel`. It is invoked in its default form: the input is the cleaned, tokenized sentence and the lower-cased target word, with no part-of-speech argument. It returns the synset whose textual description overlaps most with the context tokens, and returns no sense if the word is not in WordNet.

Comparing against this baseline isolates the contribution of the project's own design choices, summarised below.

Design element	Custom Simplified Lesk	NLTK Lesk (as invoked)
Context window	Symmetric window of half-width k, target excluded	Whole cleaned sentence supplied as context
POS handling	Tagger-derived POS filters the candidate senses	No POS argument; all senses compete
Stop words	Removed from context and signatures	Project preprocessing is not applied
Lemmatization	POS-aware lemmatization of context and signatures	Project lemmatization is not applied
Signature fields	Gloss, examples and synonyms, kept separate	Single combined description, unweighted
Weights	$\alpha = 1$, $\beta = 2$, $\gamma = 0.5$	None

Design element	Custom Simplified Lesk	NLTK Lesk (as invoked)
Tie-breaking	Example overlap, gloss overlap, then corpus frequency	Not controlled by the project
Explanation	Per-field overlap words returned	Predicted synset only

9. Evaluation Methodology

9.1 Test data format

The annotated dataset is a UTF-8 CSV file that must contain the columns `sentence`, `target_word` and `gold_synset`. The gold value is a WordNet synset name (for example `bat.n.02`). On loading, leading and trailing whitespace is trimmed and any row with an empty field is skipped. The loader fails with a clear error if a required column is missing.

9.2 Protocol

1. Load the annotated dataset and validate its columns.
2. For each row, resolve the gold string to a WordNet synset object. A row whose gold identifier is not a valid WordNet synset is reported as an error and excluded from scoring, so malformed labels never silently count as model failures.
3. Run the custom model and the NLTK model on the same sentence and target.
4. Convert each prediction to a synset name and test equality with the gold synset name (exact match). A missing prediction counts as incorrect.
5. Aggregate counts and write one result row per valid instance to the output CSV, including the predicted sense, the score, the overlap words and the correctness flags of both models.

9.3 Metrics

Let N_{valid} be the number of instances that survive step 2 and let δ_j^M equal 1 if model M predicts the gold synset on valid instance j and 0 otherwise; the sum below runs over all valid instances. Accuracy is

$$Acc(M) = \left(\sum_j \delta_j^M / N_{valid} \right) \times 100\ \% \tag{13}$$

and the comparison between the two systems is the accuracy difference in percentage points:

$$\Delta Acc = Acc(Custom) - Acc(NLTK) \tag{14}$$

Because both systems return exactly one sense for every instance that has a candidate, coverage is effectively 100 % and precision, recall and F1 all coincide with accuracy under the exact-match criterion. This is why a single accuracy figure is sufficient for the headline comparison. For finer analysis, accuracy can additionally be computed per target word, where J(w) is the set of valid instances whose target is w:

$$Acc_w(M) = \left(\sum_{j \in J(w)} \delta_j^M \right) / |J(w)| \tag{15}$$

9.4 Test data design

The test set is built around well-known ambiguous words (for example *bank*, *bat*, *crane*, *match*, *plant*, *spring*, *watch*, *rock*, *light*, *bark*) with several sentences per word, each constructed so that the surrounding context supports one reading. A set of this kind tests whether the algorithm uses context at all, but it is small and does not represent natural caption text. The prototype raw-transcript file serves as a source of realistic sentences from which new test instances can be drawn.

9.5 Annotation quality as a methodological requirement

Accuracy is defined relative to the gold label, so it can never be more reliable than the annotation. Two safeguards are built into the method: invalid synset identifiers are detected and excluded (Section 9.2, step 2), and the number of excluded rows is visible as the difference between the dataset size and N_{valid}. A further requirement, which the software cannot check automatically, is that every valid identifier must also denote the *intended* sense. In WordNet the synset names *bank.n.01* and *depository_financial_institution.n.01* denote different concepts (the former is the sloping-land sense), so a gold label must be chosen by reading the gloss, not by assuming the first sense is the commonest reading. All comparisons between model versions must use the same verified label set.

10. Complexity Analysis

Let n be the number of tokens in a sentence, k the window half-width, $m = |\Sigma(w, \pi)|$ the number of candidate senses and L the average signature size (tokens in gloss, examples and lemma names of one synset).

- **Preprocessing:** tokenization, tagging and lemmatization are O(n); window extraction is O(k).
- **Signature construction:** $O(m \cdot L)$ per instance, dominated by tokenizing and lemmatizing the WordNet text of each candidate.
- **Scoring:** each intersection of hash sets costs $O(\min(|C|, |D|))$, so scoring all candidates is $O(m \cdot \min(|C|, L))$ with $|C| \leq 2k$.
- **Ranking:** sorting m candidates is $O(m \log m)$; in practice m is below about 30.

The total cost per target word is thus $O(n + m \cdot L)$, independent of the vocabulary size and with no training phase, no GPU requirement and a memory footprint limited to the WordNet database. By contrast, original Lesk applied to a window of q ambiguous words must consider on the order of $\prod_j m_j$ sense combinations, which is why the simplified variant is the practical choice for real-time use.

Signatures depend only on the synset and the POS hint, not on the sentence. A production deployment can therefore pre-compute and cache them, reducing run-time work to context extraction and set intersections.

11. Design Rationale, Limitations and Extensions

11.1 Design rationale

- **Interpretability over opacity.** Every decision is explained by a short list of shared words, which allows errors to be diagnosed by inspection.
- **Modularity.** Preprocessing, model and evaluation are independent layers, so a new model can be added by implementing one `disambiguate(sentence, target_word)` method.

- **Controlled evidence.** Only gloss, example and synonym text are used. Expanding every signature with hypernym and hyponym glosses would increase recall of matches but also inject much unrelated vocabulary, so such expansion is deliberately not part of the core score.
- **Determinism.** The lexicographic key in Equation (10) guarantees the same output for the same input.

11.2 Limitations

- **Exact lexical matching.** Related words that are not identical after lemmatization (*guitar* and *instrument*, *music* and *rock*) do not contribute to the score. This is the main source of missed senses and of the all-zero fallback case.
- **Sparse signatures.** Glosses are short and many synsets have no example sentence, so overlap counts are small and ties are common.
- **Short captions.** A sentence with few content words gives few candidates for overlap; the decision then rests on frequency.
- **Exact target matching.** The target is located by an exact token match, so an inflected occurrence (for example *bats* for target *bat*) is not found unless the inflected form is supplied as the target.
- **Tagging on cleaned text.** POS tagging is run on lower-cased, punctuation-free text, which can reduce tagger accuracy for words that depend on capitalisation or punctuation; a wrong POS restricts the candidate set to the wrong category.
- **Repeated target words.** Only the located occurrence is excluded from the context; further occurrences of the same word remain in C.
- **Resource dependence.** Coverage and granularity of senses are those of WordNet, which is known to be very fine-grained.

11.3 Optional extensions of the scoring function

Two further evidence types are described as design options for the system (phrase evidence and a small semantic-similarity bonus). They are not part of the scoring function implemented in `models.py` at present, which is Equation (9); they are specified here so that they can be added without changing the architecture.

Phrase matching. Let $\Phi(C)$ be the set of two-word and three-word sequences in the context and $\Phi(s)$ the corresponding sequences in the sense signature (for example *river bank*, *bank account*). A phrase match is stronger evidence than two isolated word matches.

Controlled semantic similarity. A WordNet path-based similarity between context words and the candidate synset can be used as a small secondary bonus. Using it only as a bonus prevents it from overpowering lexical overlap. The extended score would be

$$Score^{+}(s) = Score(s) + \delta \cdot |\, \Phi(C) \cap \Phi(s) \,| + \eta \cdot sim(C, s) \tag{16}$$

with small non-negative constants δ and η chosen so that the bonus terms cannot exceed the contribution of a single gloss match.

11.4 Future directions

- **Confidence estimation.** The margin between the best and the second-best candidate, $\Delta_{conf} = \text{Score}(s_1) - \text{Score}(s_2)$, indicates how decisive the evidence was. Low margins can be flagged or passed to a stronger model.

- **Hybrid scoring.** Combine lexical overlap with the cosine similarity of contextual embeddings from a pre-trained transformer (for example BERT) between the context and each gloss: $\text{Score}_{\text{hyb}} = \lambda \cdot \text{Score} + (1 - \lambda) \cdot \cos(\mathbf{e}_C, \mathbf{e}_s)$.
- **Real-time caption integration.** Insert the module between the speech-to-text output and the translation engine, so that the chosen synset constrains lexical choice in the target language.
- **Multilingual mapping.** Use the Open Multilingual WordNet, which is already downloaded by the preprocessing layer, to map the chosen synset to lemmas in the target language.
- **Larger benchmarks.** Evaluate on standard WSD resources such as SemCor, the Senseval and SemEval English all-words tasks, or the Unified WSD Evaluation Framework, to obtain results comparable with published systems.

Appendix A: Notation

Symbol	Meaning
X, n	Sentence as a token sequence and its length
w, i	Target word and its token position
k	Context window half-width (tokens on each side)
C	Final context set after windowing, stop-word removal and lemmatization
π	Part of speech of the target (n, v, a, r) or undefined
Σ(w, π)	Candidate WordNet synsets of w, optionally POS-filtered
D(s), E(s), Y(s)	Normalised token sets of the gloss, examples and lemma names of synset s
o_D, o_E, o_Y	Field-wise overlap counts with C
α, β, γ	Weights for gloss, example and synonym overlap
f(s)	WordNet corpus frequency of the target lemma in synset s
κ(s)	Lexicographic ranking key of a candidate
$\hat{s}$	Predicted sense
N_{valid}	Number of test instances with a valid gold synset
Acc(M), ΔAcc	Accuracy of model M and accuracy difference between models

Appendix B: Configuration Summary

Parameter	Value	Where set
Window half-width k (model default)	7	`SimplifiedLesk.__init__`
Window half-width k (evaluation run)	5	`DEFAULT_WINDOW_SIZE` in the pipeline
POS filtering	enabled	`use_pos_filter`
Synonym evidence	enabled	`use_synonyms`
Gloss weight α	1.0	`definition_weight`
Example weight β	2.0	`example_weight`
Synonym weight γ	0.5	`synonym_weight`
Stop-word list	NLTK English	`preprocessing.py`
Lemmatizer	WordNet lemmatizer	`preprocessing.py`
POS tagger	NLTK averaged perceptron	`preprocessing.py`
Sense inventory	Princeton WordNet via NLTK	`models.py`
Baseline	`nltk.wsd.lesk`	`NLTKLeskModel`
Input dataset	`data/annotated.csv`	`DATA_FILE`
Output file	`data/annotated/evaluation_results.csv`	`OUTPUT_FILE`

The window half-width differs between the model default (7) and the evaluation pipeline (5). Any reported result should state which value was used.